perplexity

## Adaptive Drone Swarm Simulation: Technical Deep Dive

### 1. Core Behavioral Algorithms

**Cohesion/Alignment/Separation** (Boids-inspired):

```
def calculate_avoidance_force(self, swarm, obstacles):
    avoidance_force = np.array([0.0, 0.0, 0.0])
    swarm_center = np.zeros(3)
    neighbor_count = 0

    # Boids implementation
    for other_drone in swarm.drones:
        dist = np.linalg.norm(self.position - other_drone.position)
        if dist < self.avoidance_radius:
            swarm_center += other_drone.position
            neighbor_count += 1
            avoidance_force += (self.position - other_drone.position)/dist**2  # Separati

    if neighbor_count > 0:
        cohesion_force = (swarm_center/neighbor_count - self.position) * 0.1  # Cohesion
        alignment_force = (np.mean([d.velocity for d in neighbors], axis=0) - self.veloci
```

Implements Reynolds' Boids rules with inverse-square repulsion[1].

### 2. Pattern Formation Mechanics

Geometric target generation using numpy:

```
def generate_pattern_targets(self, pattern_name):
    if pattern_name == "circle":
        radius = 7.5
        return np.array([[radius*np.cos(θ), radius*np.sin(θ), 0]
                         for θ in np.linspace(0, 2π, num_drones)])
    elif pattern_name == "square":
        return np.array([[x, y, 0] for x in [-5,0,5] for y in [-5,0,5]])
```

Uses matrix operations similar to Swarm-in-Blocks' formation system[2].

### 3. Hybrid Control System

PD controller for pattern transitions:

```
def calculate_pattern_force(self):
    error = self.target_position - self.position
```

```
    derivative = -self.velocity
    return self.Kp*error + self.Kd*derivative  # PD control
```

PID controller for obstacle navigation:

```
def calculate_pid_force(self, goal_position, dt):
    error = goal_position - self.position
    self.integral += error * dt
    derivative = (error - self.prev_error)/dt
    return self.Kp*error + self.Ki*self.integral + self.Kd*derivative
```

Combines classical control theory with swarm intelligence[3].

### 4. Collision Avoidance System

3D cylinder collision detection:

```
class Pillar:
    def is_inside(self, position):
        dx, dy = position[0:2] - self.base_position[0:2]
        radial_dist = math.sqrt(dx**2 + dy**2)
        return radial_dist <= self.radius and \
               self.base_position[^2] <= position[^2] <= self.base_position[^2]+self.heig
```

Uses geometric checks for pillar avoidance[4].

### 5. Visualization Pipeline

Real-time 3D rendering with Matplotlib:

```
def plot_swarm(self):
    self.ax.clear()
    active_drones = [d for d in self.swarm.drones if not d.eliminated]
    positions = np.array([d.position for d in active_drones])
    self.ax.scatter(positions[:,0], positions[:,1], positions[:,2], c='b')
    self.canvas.draw()
```

Maintains 30+ FPS through optimized redraw cycles[5].

### 6. Performance Metrics

Success tracking implementation:

```
def calculate_run_results(self):
    successful = [d for d in self.drones if d.reached_goal]
    avg_time = np.mean([d.time_to_goal for d in successful])
    return len(successful), avg_time
```

Provides quantitative swarm performance analysis[6].

Would you like me to elaborate on any specific algorithm or demonstrate how these components interact in different simulation scenarios?

⁂


1. https://people.ece.cornell.edu/land/courses/ece4760/labs/s2021/Boids/Boids.html
2. https://www.imavs.org/papers/2024/34.pdf
3. https://github.com/matteoprata/UAV-Swarm-Simulator
4. https://github.com/lis-epfl/vmodel
5. https://munin.uit.no/bitstream/10037/23276/2/thesis.pdf
6. proposal.pdf